

Otimização de Footprint em LLVM-IR via Compressão de Huffman

Uma Abordagem Zero-Dependency para Sistemas Embarcados

[AUTHORS]: Luiz Gustavo & Hendrick Silva

[DOMAIN]: Análise de Algoritmos

[TARGET]: Sistemas Restritos (Edge / OTA)

A Restrição Físico-Lógica

O Contexto (Edge Computing)



Atualizações Over-the-Air (OTA) exigem transmissão em redes de banda ultrabaixa.
Memória Flash e RAM são **estritamente limitadas** no dispositivo final.

O Problema (LLVM-IR)



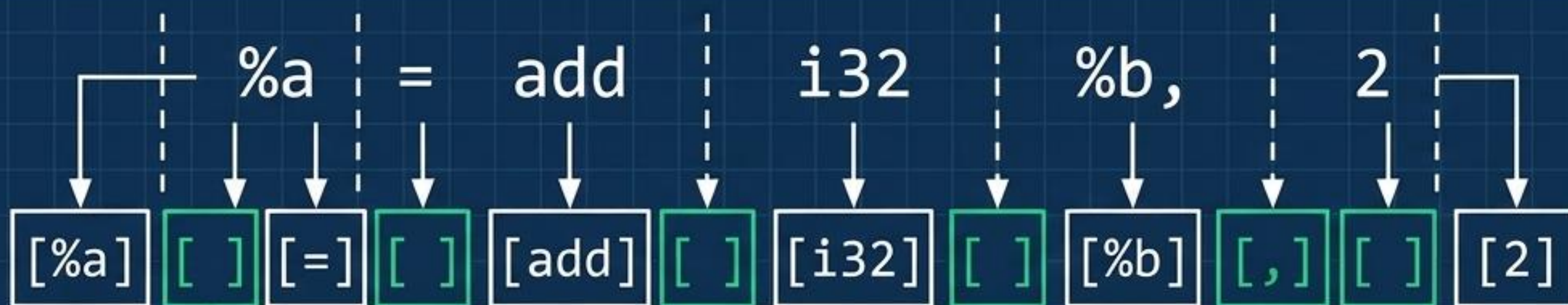
O Código Intermediário (LLVM-IR) é expressivo, mas **volumoso**. Binários pesados **drenam bateria** e **prolongam o tempo crítico** de transmissão OTA.

A **MISSÃO**: Desenvolver um compressor **sem perdas (lossless)**, **desenhado do zero e sem uso da biblioteca padrão (stdlib)**, para garantir **controle absoluto** e **determinístico** sobre a **gestão de memória**.

O Objetivo Matemático e a Invariante Lossless

$$\min L(P) = \sum f(s) \cdot |c(s)|$$

Minimizar o comprimento da sequência sob a restrição de códigos de prefixo unívocos.



A Invariante: Nenhum byte é descartado, nem mesmo espaços em branco. A concatenação sequencial exata dos tokens reconstrói o arquivo .ll original bit a bit.

A Arquitetura do Algoritmo (Pipeline)



Engenharia de Estruturas: Customizado vs. Padrão

Dimensão	Abordagem Padrão (stdlib)	Nossa Implementação (Zero-Dep)
Gestão de Frequências	Dinâmica, alocações de memória <u>imprevisíveis</u>	Hash Table <u>Estática</u> ($m=1024$). Fator de carga $\alpha < 1$. Custo de inserção: <u>$O(1)$</u> .
Fila de Prioridade	Árvores complexas, <u>alto overhead</u> de ponteiros opacos	Min-Heap Binário sobre Array. Operações de sift-up/down <u>in-place</u> . Custo: <u>$O(\log k)$</u> .
Controle de Memória	<u>Caixa-preta (Black-box)</u>	<u>Determinístico</u> . <u>Previsibilidade total</u> de footprint, ideal para restrições de Edge Computing.

O núcleo de performance: A integração direta entre a Hash Table e o Min-Heap em array viabiliza o algoritmo guloso em ambientes severamente limitados.

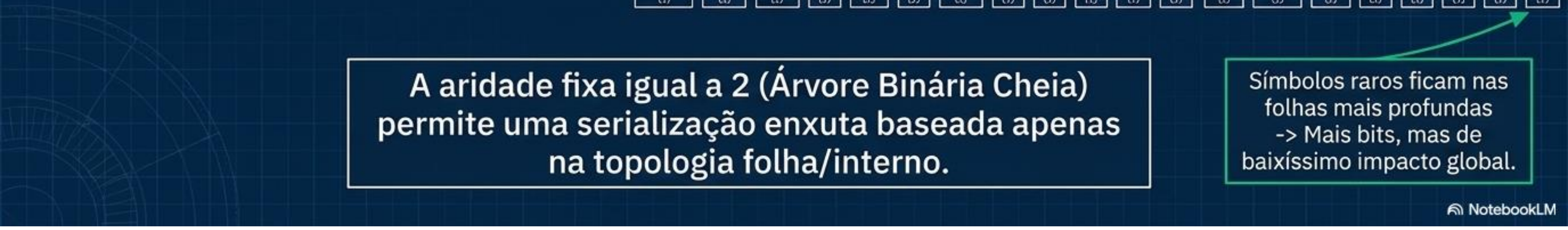
O Motor de Complexidade



parser_tokenize :	$O(n)$
ft_add :	$O(1)$ média
mh_push / pop :	$O(\log k)$
huff_build_tree :	$O(k \log k)$
codes_build(DFS) :	$O(k)$

n = total de bytes no arquivo bruto. k = total de tokens únicos no alfabeto (onde $k \ll n$).

ex:



A aridade fixa igual a 2 (Árvore Binária Cheia) permite uma serialização enxuta baseada apenas na topologia folha/interno.

Símbolos raros ficam nas folhas mais profundas
-> Mais bits, mas de baixíssimo impacto global.

Layout do Arquivo Binário Serializado

Metadados (Cabeçalho)

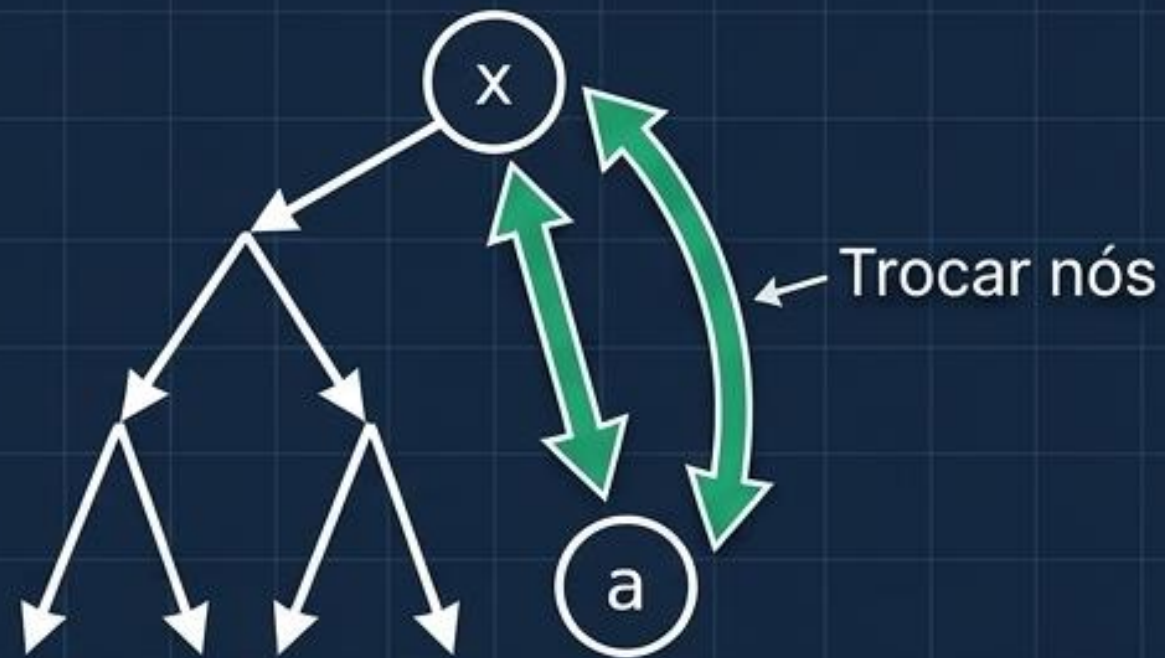
[HUF1]	[# TOKENS]	[TAM. DICT]	[DICIONÁRIO SERIALIZADO]	[FLUXO DE BITS]
4 bytes. Assinatura mágica	u32. Quantidade autodelimitadora	u32. Tamanho do dicionário	Pré-ordem. Usa 1 byte de tag (1=folha, 0=interno)	Dados comprimidos. MSB para LSB + padding

Insight de Design:

A reconstrução não requer delimitadores artificiais de fim-de-arquivo no fluxo de bits. O cabeçalho dita a topologia, e a topologia dita a leitura matemática estrita do dicionário.

Geometria da Otimalidade: A Prova Visual

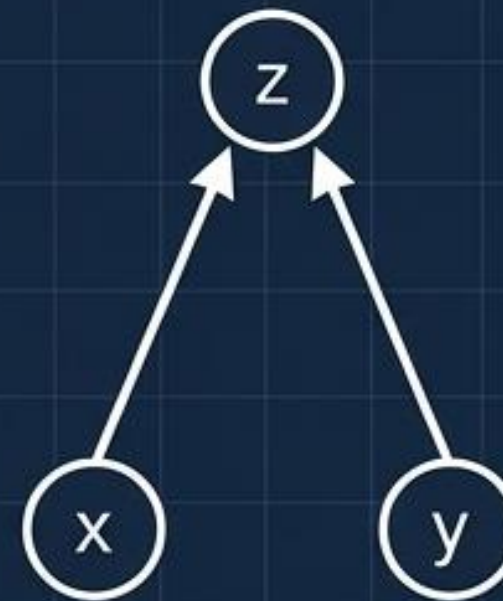
Lema 1: Escolha Gulosa



$$\Delta = (f(x) - f(a)) \cdot (d_a - d_x) \leq 0$$

Trocar o nó menos frequente para a posição mais profunda nunca aumenta o custo global.

Lema 2: Subestrutura Ótima



$$f(z) = f(x) + f(y)$$

O problema é monotonicamente reduzido até a raiz. Por indução geométrica, a árvore final gerada é garantidamente a de menor custo possível.

Assimetria Computacional: Encode vs. Decode

Custo: $O(n \cdot k)$

Compressão - Encode

A fase de codificação é computacionalmente pesada, dominada pela busca linear repetitiva do símbolo no dicionário para gerar a sequência binária correspondente.

Custo: $O(B)$

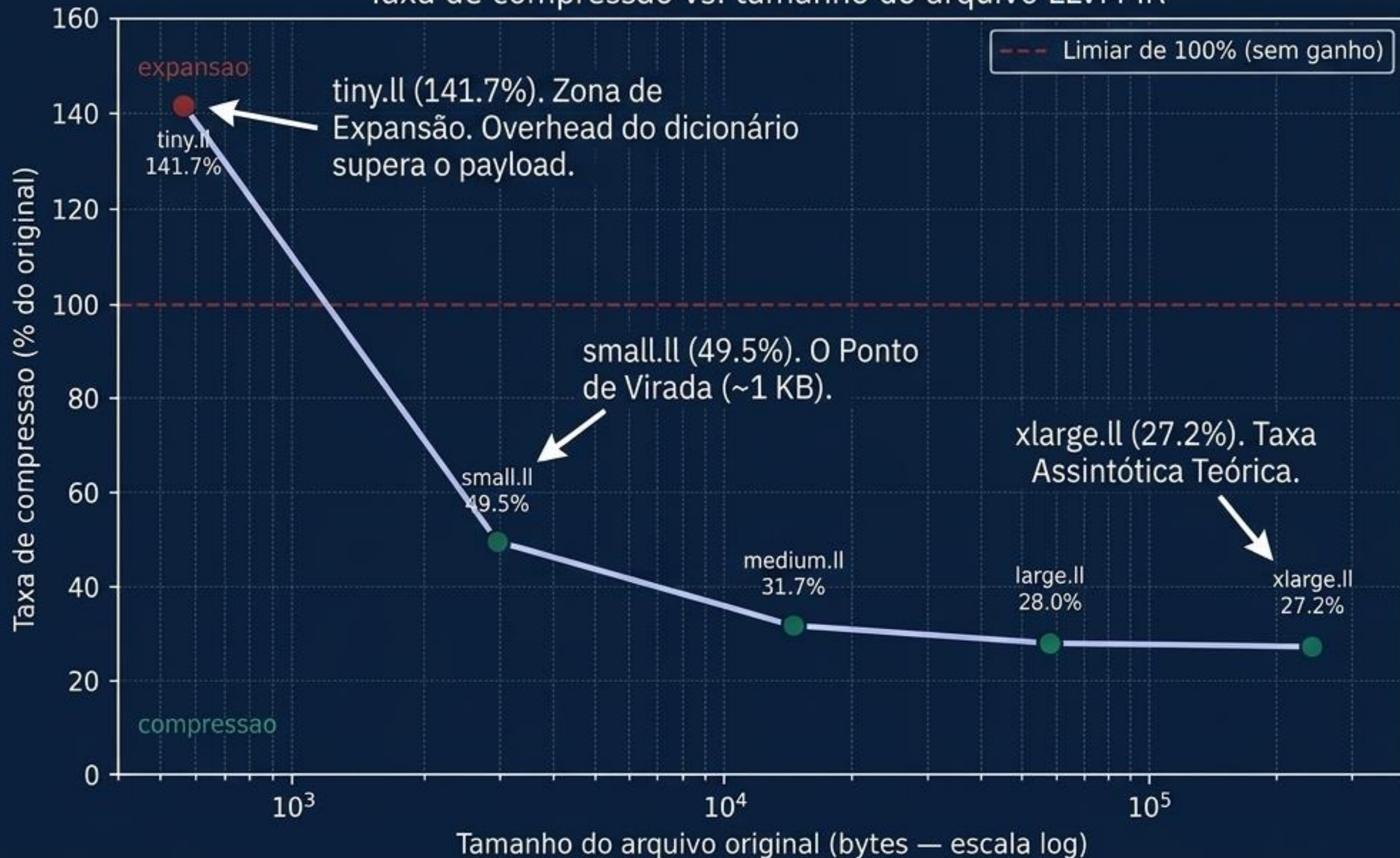
Descompressão - Decode

Extremamente leve e veloz. Consiste em um único passo lógico por bit lido (onde B = total de bits), navegando puramente na topologia da árvore já reconstruída em memória.

ARQUITETURA WRITE-ONCE, READ-MANY: O custo computacional é pago integralmente e antecipadamente no servidor (Encode). O dispositivo de borda (Edge) executa apenas leitura linear leve (Decode). Os testes experimentais de **round-trip** resultaram em integridade 100% idêntica (lossless verificado via diff).

A Fronteira de Eficiência Empírica

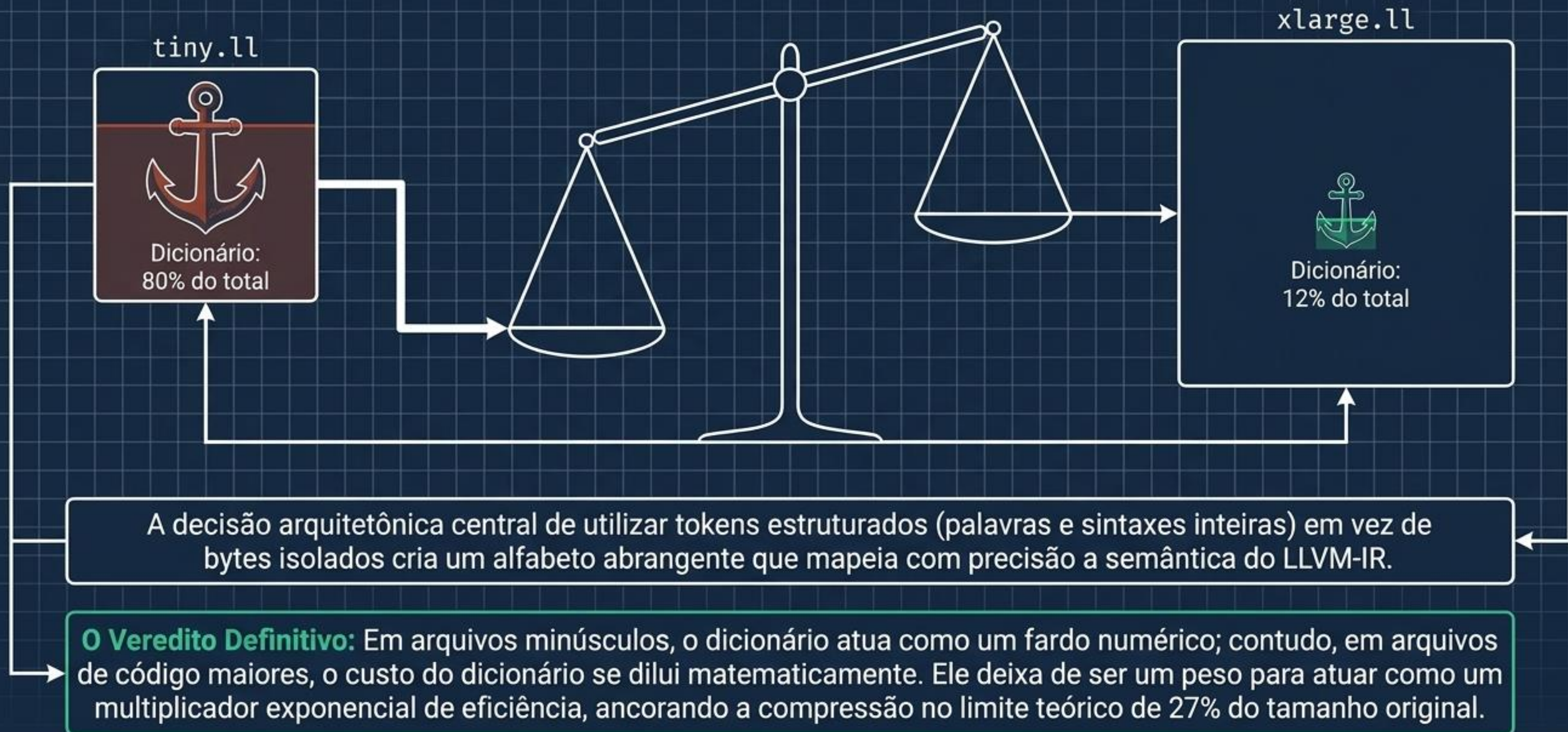
Taxa de compressão vs. tamanho do arquivo LLVM-IR



A performance empírica revela a dualidade do sistema.

Arquivos minúsculos sofrem com o peso fixo da estrutura matricial (640 bytes de metadados mínimos). No entanto, ao ultrapassar a marca de ~1KB, a curva quebra drasticamente a favor da eficiência de rede e da mitigação térmica no dispositivo.

Síntese: A Lei da Amortização do Dicionário



Vetores de Otimização e Trabalhos Futuros

1. Granularidade Dinâmica (N-Grams)

Agrupar sequências repetitivas de instruções LLVM-IR. Utilizar uma variação do **Problema da Mochila** (Programação Dinâmica) para avaliar se o ganho de compressão de um padrão justifica o seu **custo permanente** no armazenamento do dicionário.

2. Otimização de Busca no Encode: $O(1)$

Substituir a atual travessia e busca linear (símbolo $\rightarrow$ código) por uma **Hash Table secundária** exclusiva para a etapa de compressão em servidor, reduzindo a complexidade temporal da codificação de $O(n \cdot k)$ para $O(n)$ em média.

3. Compactação Extrema de Metadados

Reduzir o campo descritor do tamanho dos símbolos no dicionário de uma arquitetura de 4 bytes (u32) para apenas **1 byte rígido**, **suprimindo drasticamente o overhead fixo** e movendo o Ponto de Virada para arquivos ainda **menores que 1 KB**.